Assignment 1 Design and Development

Appointment Booking Microservices

Scalable Services (SEZG583) | MTech | 13 marks

Submission details	Current information
Application	Appointment booking using four independently runnable Python and FastAPI services
Team	Team 8; [add student names and IDs; maximum four students]
Repository	[Add accessible GitHub repository URL]
Demo video	[Add Google Drive link to the 8-10 minute video]
Report date	26 October 2026

Report status

This report records the implementation and locally captured run results in the repository. Team names, individual contribution and reflection statements, repository URL, and video link are marked for the team to supply because those facts cannot be inferred from the code.

The application demonstrates a booking transaction across inventory, payment, and notification services. The evidence runs show capacity protection, replay safety, compensation after a payment outage, and a successful end-to-end booking.

1. Problem and scope

The application books limited-capacity appointment slots. A client asks the Booking service to reserve a slot, charge a simulated payment, and record a notification. The design is intended to demonstrate service ownership and correct behavior when requests compete, repeat, or encounter a downstream failure.

Assumptions and scope

- Each slot has a finite capacity. Inventory owns the capacity and reservation state.

- The Booking service coordinates the workflow synchronously over REST. Payment and notification are educational mocks; no real payment or message is sent.

- Authentication, user interface, real payment integration, and production deployment are out of scope.

- SQLite keeps local setup small. The report treats the implementation as a teaching demo, not a high-throughput production deployment.

Business rules

ID	Rule	Implementation and test
BR1	A slot cannot be over-allocated; remaining capacity must not become negative.	Inventory uses an atomic conditional update. Test: `test_br1_inventory_never_overallocates_last_slot`; concurrency evidence: 20 parallel calls.
BR2	A booking needs customer ID, slot ID, positive amount, and an Idempotency-Key.	Pydantic validation and required-header check. Test: `test_br2_requires_valid_fields_positive_amount_and_idempotency_key`.
BR3	Same key and same payload replay the saved result; a changed payload using that key is rejected.	Canonical fingerprint and stored response. Test: `test_br3_same_key_replays_one_result_and_changed_payload_conflicts`; Challenge 2 evidence also covers simultaneous calls.
BR4	When payment fails after reservation, Booking attempts to release the reservation.	Booking requests Inventory compensation. Test: `test_br4_payment_failure_releases_inventory_reservation`; Challenge 3A outage run.
BR5	A notification is attempted only after payment succeeds.	Booking invokes notification after a successful charge. Test: `test_br5_notification_is_attempted_only_after_successful_payment`.

2. Service decomposition

The boundaries follow business capabilities. Booking owns the workflow and booking records; Inventory owns slot capacity; Payment and Notification own their mock records. No service queries another service's SQLite database.

Service	Responsibility	Owned data	Port
Booking	Coordinates reserve, charge, notification, replay, and compensation.	Bookings and idempotency responses.	8000
Inventory	Atomically reserves or releases appointment capacity.	Slots and reservations.	8001
Payment mock	Records simulated charges and deduplicates charge keys; can be stopped or configured to fail.	Payments.	8002
Notification mock	Records a simulated post-payment notification.	Notifications.	8003

Why these boundaries

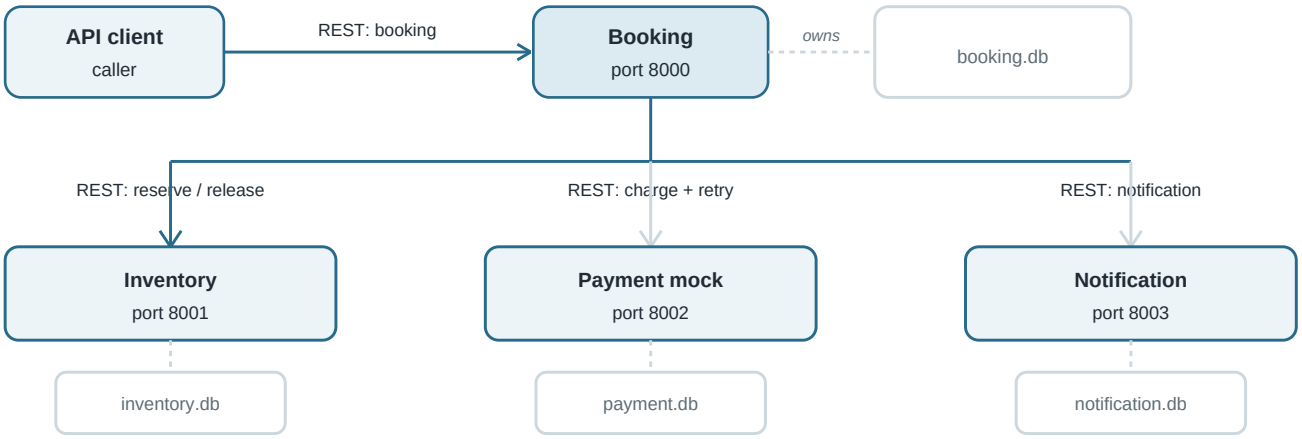
Inventory changes capacity independently of booking orchestration. Payment and notification are external-style dependencies with their own data and failure modes. Booking owns the transaction sequence because it must coordinate the reservation, charge, notification, and compensation. The design avoids four generic database wrappers by placing actual business rules in the services.

Alternative considered

A single application and database would make atomic transactions simpler, but it would not demonstrate database-per-service ownership or independent failure. Splitting each table into its own service would add calls without a distinct business responsibility. The selected four-service split is based on the appointment workflow and its independent dependencies.

3. Architecture and transaction sequences

The services communicate over REST and pass X-Request-ID between downstream calls. FastAPI provides a per-service OpenAPI document at `/openapi.json` and interactive documentation at `/docs`.



Each service owns its database; cross-service references are IDs only.

Figure 1. Service components, REST calls, and separate data stores.

Happy path

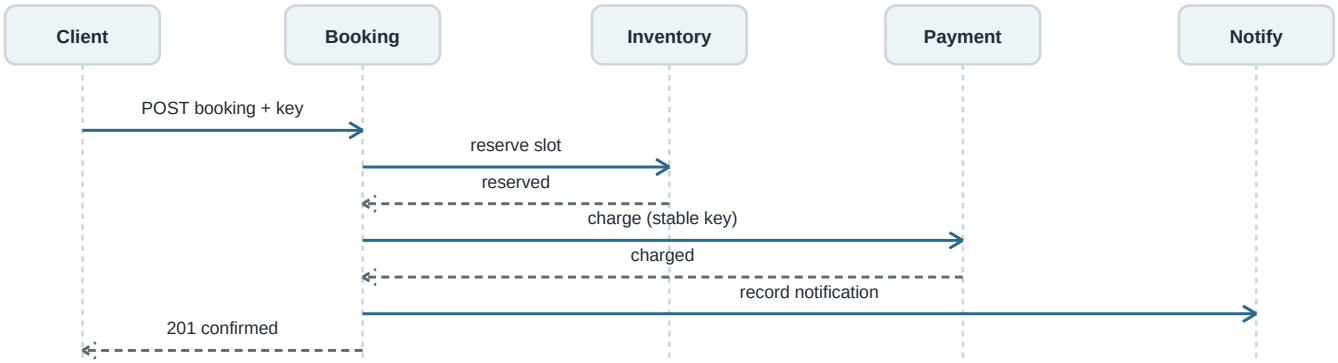


Figure 2. Successful booking: reserve, charge, notify, and confirm.

Payment failure path

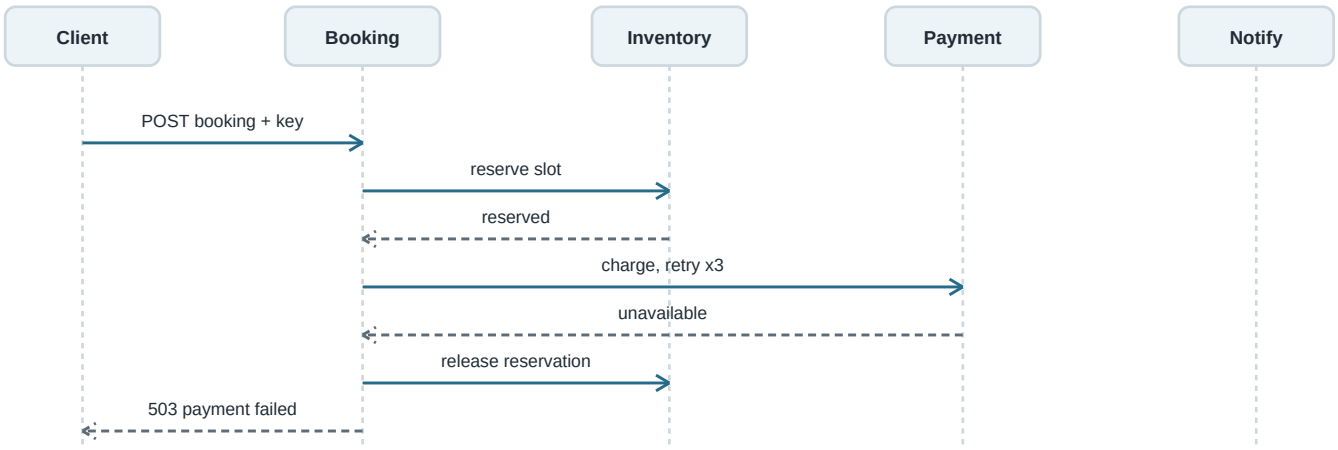


Figure 3. After bounded payment retries fail, Booking attempts to release the reserved slot and returns HTTP 503.

4. Data design and ownership

Each service owns its SQLite database. Cross-service associations use IDs in payloads; there are no cross-database joins or foreign keys. Each database can be replaced independently, subject to the workflow's need to reconcile partial failures.

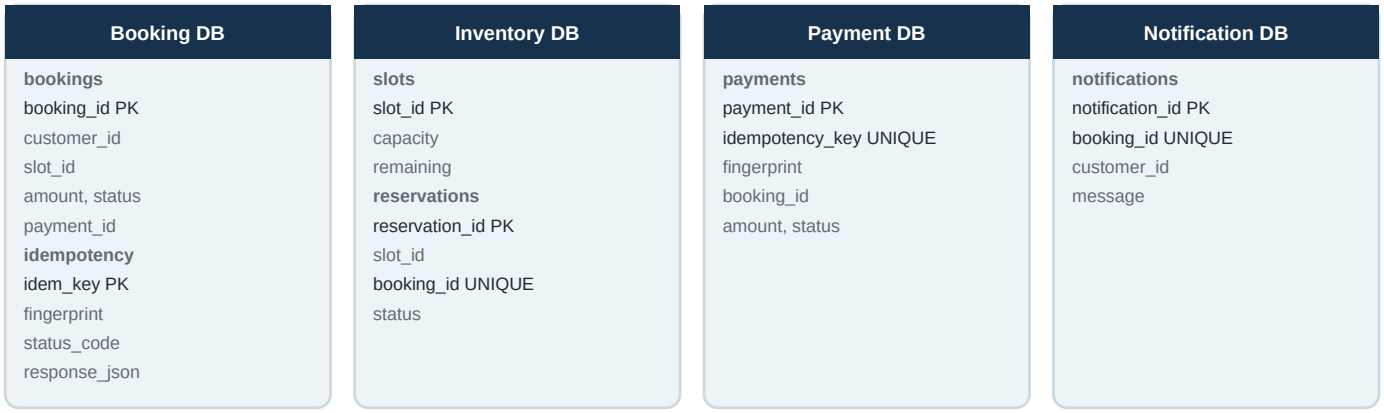


Figure 4. Per-service entity relationships. PK and UNIQUE identify local constraints; cross-service IDs are not foreign keys.

Ownership notes

- Booking stores the request fingerprint and the response associated with an idempotency key.
- Inventory stores local slot capacity and reservation status. Its SQL update only decrements a positive remaining value.
- Payment stores a unique idempotency key and payload fingerprint for each mock charge.
- Notification stores one record per booking ID, using a local uniqueness constraint to avoid duplicate records.

The local SQLite files are for development and evidence runs. Production deployment would require managed database configuration, credentials, backups, and concurrency controls appropriate to multiple service instances.

5. API design

All business endpoints are versioned under `/api/v1`. FastAPI validates request bodies and publishes OpenAPI. The main booking endpoint requires the `Idempotency-Key` header; booking propagates the request ID to downstream calls.

Service	Method and path	Purpose and key result
Booking	POST `/api/v1/bookings`	Create or replay a booking; 201 on confirmation, 409 for key/payload conflict, 503 on payment failure.
Booking	GET `/api/v1/bookings/{booking_id}`	Read a booking; 200 or 404.
Inventory	GET `/api/v1/slots`	List slot capacity and remaining units.
Inventory	POST `/api/v1/slots/{slot_id}/reserve`	Reserve one unit; 409 when full, 404 for unknown slot.
Inventory	POST `/api/v1/slots/{slot_id}/release`	Compensate a reservation after payment failure.
Payment mock	POST `/api/v1/payments/charge`	Record or replay a simulated charge; supports a failure mode.
Notification mock	POST `/api/v1/notifications`	Record a notification after successful payment.
All services	GET `/health`; GET `/openapi.json`; GET `/docs`	Health and generated API documentation for each service.

Example request

`POST http://127.0.0.1:8000/api/v1/bookings` with `Idempotency-Key: demo-001` and JSON body `{"customer_id":"cust-1","slot_id":"slot-1","amount":25.00}`. A confirmed response includes booking, reservation, and payment IDs plus notification status.

Error model

Application errors use a JSON envelope containing `error.code`, `error.message`, and `error.request_id`. FastAPI request-validation failures use FastAPI's standard validation response, so validation errors are not fully normalized to the shared error envelope yet.

OpenAPI endpoints

- Booking: `http://127.0.0.1:8000/openapi.json`
- Inventory: `http://127.0.0.1:8001/openapi.json`
- Payment mock: `http://127.0.0.1:8002/openapi.json`
- Notification mock: `http://127.0.0.1:8003/openapi.json`

6. Engineering challenges and evidence

Challenge 1: concurrency control

Inventory executes `UPDATE slots SET remaining=remaining-1 WHERE slot_id=? AND remaining>0` inside a write transaction. This makes the decrement conditional at the database boundary. A plain application-level check followed by a later update can let multiple requests observe the same final unit before a write occurs.

- Recorded run: 20 parallel requests against `slot-2` with capacity 1; exactly 1 HTTP 201 and 19 HTTP 409 responses.

- Final database check: capacity 1, remaining 0, one active reservation.

- Evidence files: `evidence/challenge-1-concurrency/concurrency-run.txt`, `final-database-state.txt`, and `inventory-service.log`.

Challenge 2: idempotency

Booking stores a canonical JSON fingerprint, HTTP status, and serialized response under a unique idempotency key. SQLite `BEGIN IMMEDIATE` serializes concurrent duplicate-key handling in this demo. A changed payload with an existing key returns HTTP 409. The payment mock also stores unique charge keys.

- Sequential exact replay returned the same response and `Idempotency-Replayed: true`; the changed payload returned 409.

- Twenty simultaneous identical requests returned one identical booking result; one request performed the work and 19 responses were marked as replays.

- The run used two distinct keys and recorded two bookings, reservations, payments, and notifications total, one per key.

- Evidence files: `evidence/challenge-2-idempotency/idempotency-run.txt`, `booking-service.log`, and `database-record-counts.json`.

Challenge 3A: resilience

The selected optional challenge uses a one-second downstream timeout, three total payment attempts, exponential backoff, and jitter. On persistent failure, Booking attempts a compensating inventory release and returns HTTP 503. The payment key is derived from the booking ID so retries for one booking use the same key.

- With payment stopped, the request returned 503 after two retry delays; the booking was stored as FAILED.

- After compensation, the reservation was RELEASED and slot capacity returned to 5. After payment restarted, a new key succeeded with HTTP 201 and one payment record.

- Evidence files: `evidence/challenge-3a-resilience/resilience-run.txt`, `booking-service.log`, and `database-states.json`.

All run outputs are local evidence, not a claim of production load capacity. Screenshots of the commands and outputs still need to be added to the evidence folders.

7. Testing and results

Business-rule tests use Python's standard-library `unittest` runner and isolated SQLite databases. The end-to-end script exercises the live service process chain and verifies persisted records in each database.

Test	Observed result	Evidence
BR1-BR5 unit tests	5 tests passed; capacity, validation, replay/conflict, compensation, and notification ordering covered.	`evidence/tests/business-rule-tests.txt`
Main transaction E2E	Create booking returned 201; retrieve returned 200; one confirmed booking, reservation, payment, and notification were present.	`evidence/tests/e2e-run.txt` and database checks JSON
Concurrency challenge	1 success and 19 rejections; final slot state consistent.	`evidence/challenge-1-concurrency/`
Idempotency challenge	Exact replay, changed-payload conflict, and 20 simultaneous duplicates verified.	`evidence/challenge-2-idempotency/`
Resilience challenge	Payment outage retried and failed gracefully; reservation released; recovery succeeded.	`evidence/challenge-3a-resilience/`

Test command

From the project root: `python -m unittest discover -s tests -p test_business_rules.py -v`. Install the booking and inventory service requirements and `tests/requirements.txt` first. The current evidence includes raw text logs and database checks; screenshot evidence remains to be captured.

8. Design decisions and trade-offs

Decision	Options considered	Choice and rationale	Trade-off
REST versus broker	Synchronous REST or asynchronous broker events.	REST keeps the classroom flow visible in API calls and satisfies the assignment. Each service exposes a direct HTTP contract.	The booking path depends on downstream availability and has a synchronous chain.
Orchestration versus choreography	Booking coordinates steps or services react to events.	Booking orchestration makes reserve, charge, notify, and compensation order explicit and easy to trace.	Workflow logic is concentrated in Booking; partial failure still requires reconciliation.
SQLite versus server database	Local SQLite or a managed relational database.	SQLite reduces setup friction and supports atomic conditional updates for the demo.	A shared-file SQLite database is not suitable for multi-instance production scale or operational resilience.
Compensation versus distributed transaction	Attempt release after payment failure or require distributed transaction coordination.	Compensation avoids claiming cross-service ACID behavior and demonstrates a saga-like action.	The compensation can fail, and a process crash can leave an intermediate state.

9. Limitations and future work

- The booking service holds a local SQLite write transaction while calling downstream services. This serializes same-key requests but harms throughput and should be replaced by durable workflow state for production.

- The services cannot share a distributed ACID transaction. A crash between reservation, charge, and local persistence can require reconciliation.

- Notification is best-effort after payment and has no outbox or retry worker.

- Authentication, production observability, deployment, and operational secrets management are out of scope. Configure per-service credentials and database services before a production rollout.

- The current SQLite files are separate but do not have separate database credentials. The Uvicorn ports are supplied on the startup command rather than read from an application port environment variable; both are gaps against the assignment's operability checklist.

- A fresh clone has not yet been verified from the README, and the repository URL is not available in this workspace.

- Future work includes explicit booking states, durable saga/outbox processing, metrics and tracing, and a server database with managed backups and access control.

10. Submission links

GitHub repository: [Add accessible URL]. Google Drive folder containing the video and other documents: [Add link]. Demo video: [Add 8-10 minute recording link].

Appendix A. Team contributions

Complete this table using the team's actual work and repository history. Do not leave placeholder entries in the submitted copy.

Member	Owned work	Evidence (commits, files, demo)
[Name and ID]	[Service or task; specific contributions]	[Commit links or evidence]
[Name and ID]	[Service or task; specific contributions]	[Commit links or evidence]
[Name and ID]	[Service or task; specific contributions]	[Commit links or evidence]
[Name and ID]	[Service or task; specific contributions]	[Commit links or evidence]

Appendix B. Individual reflections

Each student should add an individual half-page reflection describing what they built, the hardest problem they solved, and what they would change. These personal statements are intentionally not fabricated here.

[Student name and ID] - [Add reflection]

[Student name and ID] - [Add reflection]

[Student name and ID] - [Add reflection]

[Student name and ID] - [Add reflection]

Appendix C. AI-use declaration

During preparation of this report and evidence package, OpenAI ChatGPT/Codex was used to review the assignment brief and repository, help add business-rule tests, create repeatable evidence-runner scripts, execute local checks, and draft report text. The team must review the generated material, ensure it understands and can explain the code, and add any AI tools used by team members for other work. The team remains responsible for the final submission and its accuracy.

Appendix D. Evidence index

- Concurrency: `evidence/challenge-1-concurrency/README.md`.
- Idempotency: `evidence/challenge-2-idempotency/README.md`.
- Resilience 3A: `evidence/challenge-3a-resilience/README.md`.
- Unit and end-to-end tests: `evidence/tests/README.md`.

Add the required timestamped screenshots to each evidence folder after capture.